

Characterization of Data Generating Neural Network Applications on x86 CPU Architecture

Antara Ganguly

A. Ganguly^{*}, S. Balachandran[†], A. Nori[†], V. Singh^{*}, S. Subramoney[†]

^{*}Indian Institute of Technology Bombay

[†]Processor Architecture Research Lab, Intel Labs

ISPASS 2020

Outline

- 1 Characterization of Data Movement Issues on CPU
- 2 Methodology
- 3 Results
- 4 Wrapping up

Characterization of Data Movement Issues on CPU

Generative NN Applications Characterization

- existing DNN studies base their research on classification tasks,
- recent class of NN applications - generate data from scratch,
- eg. face swapping, anime generation, motion transfer, super resolution, music note generation and so on,
- evident departure from traditional uses of NNs,
- do not have clear boundaries between training and inference,
- may exercise subset of the data or a subset of the layers in their task,
- representational character of a wider application base.

Memory-boundedness in Generative NN Applications

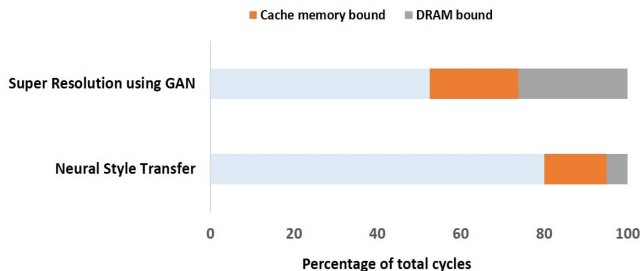


Figure: Memory-bound cycles in generation based application

- significant proportion of cycles are being spent on data fetches,
- efficiency of data movement - major component in overall performance of an end-to-end run of these application.

Methodology

Granularity of Characterization

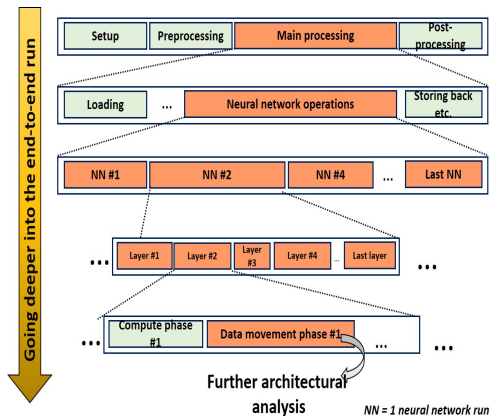


Figure: Top-down approach to characterization of neural network workloads

Experimental Setup

System used	Intel Xeon CPU E5-2640 v3
Frequency of operation	2.60 GHz (turbo frequency off)
Sockets	2
Cores	8 cores per socket (hyperthreading off)
ISA	64-bit with Intel AVX2
L1D / L1I / L2 / LLC	32 KB / 32 KB / 265 KB / 20 MB
DRAM bandwidth	84.15 GB/s average on runtime

Note : Experimental runs tied to single socket

- to avoid inter-socket migration related performance overheads.

Performance measurement

$$\text{Peak MACs/cycle} = \text{Number of AVX ports} \times \frac{\text{AVX width}}{\text{Data type width}}$$

- for AVX2, number of ports / width : 2 / 256,
- workloads restricted to using floating point data-type,
- therefore, **peak MACs/cycle is 16.**
- multi-thread scaling experiments
 - peak MACs/cycle : 32, 64 and 128 for 2-, 4- and 8-thread experiments,
 - MACs/cycle per thread is 16 for all multi-thread configuration.

Results

Poor Performers in Convolution layers

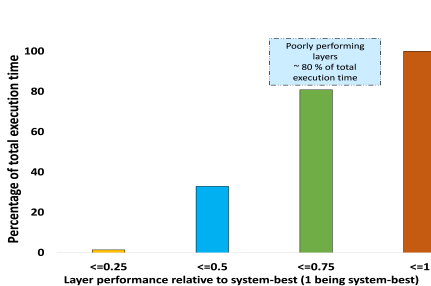


Figure: Neural Style Transfer convolution layers

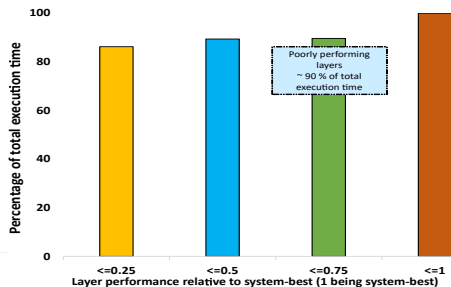
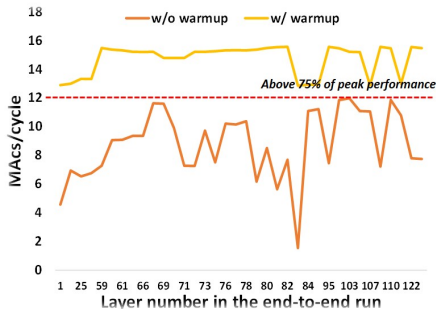


Figure: Super Resolution GAN convolution layers

Warm-up experimentation

- analyzing dip in the performance of a layer,
- experiment with data movement based system conditions without impacting the computational data reuse,
- code harness written using MKLDNN constructs,
- data handles for compute and data movement parts separated without impacting algorithmic correctness,
- synthetic readiness/availability of data - harness augmented with **warm-up** phase before each convolution primitive
- warm-up phase runs through complete address space of inputs and weights
- aim - reducing stall cycles required for loading them inside the convolution primitive.

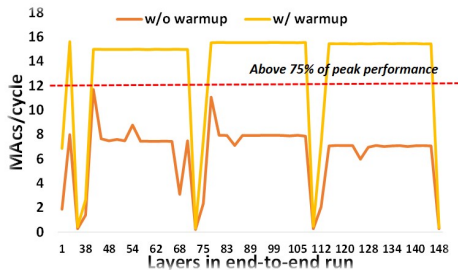
Impact of Warming-up



Layers that had shown a dip in the actual run are the first layers of the distinct phases of the neural network or are part of the stylized generation phases.

Figure: Warming-up Neural Style Transfers poorly performing convolution layers

Impact of Warming-up (contd.)



Layers with 3X3 as their kernel size benefit from warm-up, however, the remaining layers do not show significant jump in performance.

Figure: Warming-up Super Resolution GAN poorly performing convolution layers

Scope for Warm-up in Algorithm

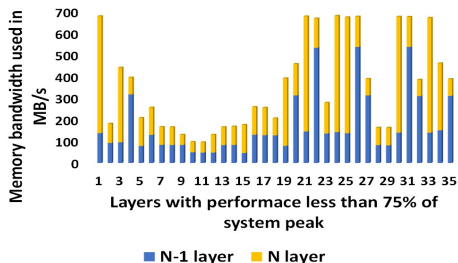


Figure: Bandwidth stack of N and N-1 layers

- examine if room in bandwidth can be used for forcefully warming up poorly performing layers beforehand,
- each layer N that saw a benefit by using warm-up, we stack the average bandwidth used by layer N-1,
- stack height - only 13% at maximum (and 7% on average) of the total bandwidth available in the system,
- suggestion - use of available bandwidth in layer N-1 to prefetch data for layer N.

Computational issues

- data warm-up did not improve efficiency for certain layers,
- hints at different operational implementation for the kernels,
- to understand difference in convolution operation being performed in layers with varied kernel sizes,
- disassembly of the kernel code analyzed,
- instructions responsible for performing algorithmic MACs - VFMA.

VFMA instruction density

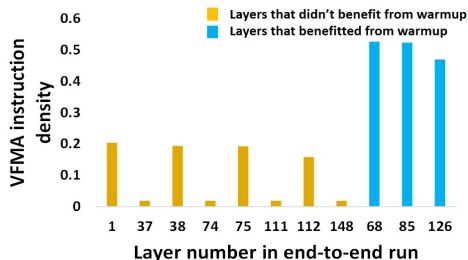


Figure: SRGAN disassembly analysis

- VFMA instructions density for non 3x3 layers $\leq 20\%$,
- most layers showing values lesser than 2%,
- other layers' VFMA instruction density - more than 45%,
- other layers use vector functional units ≥ 20 ,
- thus, inefficiency is not in cache system for execution of such layers,
- hence, they do not respond to warm-up like the 3X3 layers did.

Need for support from Kernel Library

- convolution code optimised using JIT compilation for 3X3 kernels
 - leverage vectorization of operations through available architectural features to the maximum
- other kernel sizes do not invoke this optimized version,
 - non-JIT implementation like GEMM-BLAS implementation,
 - or hand written code,
 - absence of optimized implementations for kernels that make lesser appearance in conventional CNNs
- performance drop significant,
- different instruction mapping, architectural components not optimized for MAC operations used,
- major reduction in the amount of SIMD execution,
- results in increased data movement ultimately
- library support needed to map all types of kernel sizes to the vectorization instructions in an efficient manner.

Wrapping up

Conclusions

- Characterization of generation based NN applications on CPU,
- analysis of performance bottlenecks in end-to-end runs,
- identification of DNN workloads - generation of content instead of the conventional classification tasks
- study from source code through library support to layers, further into actions/operations
- zoom into layers that fail to perform at system best,
- certain layers are not warmed up in actual workload run - contrary to what happens in layers of a standalone network,
- bandwidth analysis to understand bandwidth requirements for forced warming-up,
- layers that even after warming up do not show expected performance - library kernels analysis.

Thank you.